Project Planning & Management

1.1 Project Proposal

- **Project Name:** EduHub – Online Learning Platform
- **Domain:** E-Learning & Educational Technology
- **Executive Summary:** EduHub is a modern, scalable e-learning platform inspired by enterprise platforms like Udemy. It enables instructors to publish and structure on-demand video courses, while providing students with an interactive learning path featuring real-time progress tracking, integrated quizzes, automated certificate generation, and secure payments.
- **Core Objectives:**
    - Build a scalable, high-performance web application utilizing ASP.NET Core MVC and N-Tier Architecture.
    - Implement clean design patterns including Repository, Unit of Work, Strategy, and Observer patterns to achieve separation of concerns and maintainability.
    - Deliver seamless cloud video streaming and media asset protection using Azure Blob Storage with secure access.
    - Provide automated payment processing via Stripe and dynamic PDF certification using iTextSharp.
- **Project Scope:**
    - **In-Scope:** Multi-role authentication (Student, Instructor, Admin), interactive course builder (Sections & Lessons), secure video streaming, automated course progress calculation, quizzes and assessments, shopping cart and payment gateway integration, downloadable verified certificates, and an administrative dashboard with analytics.
    - **Out-of-Scope (Phase 1):** Real-time peer-to-peer live streaming and interactive whiteboards (reserved for future updates).

1.2 Project Plan & Milestones (4-Week Schedule)

Sprint / Week	Phase & Focus	Key Deliverables
Week 1	**Foundation & Architecture Setup**	- N-Tier architecture solution setup (Web, Services, Data) - Database modeling & Entity Framework Core Code-First configuration - Generic Repository & Unit of Work implementation - ASP.NET Identity multi-role authentication setup - Course catalog browsing with search and category filtering
Week 2	**Instructor Dashboard & Media Management**	- Comprehensive instructor portal & interactive course builder - Azure Blob Storage integration for video uploads and streaming

Sprint / Week	Phase & Focus	Key Deliverables
		- Secure media URL generation and thumbnail handling - Pricing, discount rules, and course draft/publish workflows
Week 3	**Student Experience, Assessments & Checkout**	- Video playback interface using Video.js with automated progress tracking - End-of-section interactive quiz engine with immediate grading - Stripe payment gateway integration with shopping cart - Discount coupon engine utilizing the Strategy Pattern - Course review and rating module

Sprint / Week	Phase & Focus	Key Deliverables
Week 4	**Certifications, Admin Dashboard & Deployment**	- Automated PDF certificate generation using iTextSharp with verification codes - Administrative analytics dashboard using Chart.js - Platform moderation (course review/approval workflow) - Final end-to-end integration testing and code optimization - Production cloud deployment to Microsoft Azure App Service and Azure SQL

1.3 Task Assignment & Team Roles

Role	Core Responsibilities
Full Stack Lead & Architect	Solution architecture design (N-Tier), implementing structural and behavioral design patterns, coordinating core business logic in the Service layer, and managing code reviews.

Role	Core Responsibilities
Database & Cloud Engineer	Database schema design, EF Core migrations, index optimization, Azure Blob Storage container management, and continuous deployment (CI/CD) pipelines.
Frontend & UI/UX Developer	Responsive interface implementation with Bootstrap 5, client-side interactions, video player integration (Video.js), and data visualization charts (Chart.js).
Security & Integration Engineer	ASP.NET Identity role-based access control (RBAC), Stripe checkout integration, secure token handling, and automated PDF certificate generation engine.

1.4 Risk Assessment & Mitigation Plan

Identified Risk	Impact Level	Mitigation Strategy
Media Latency & High Bandwidth Overhead	High	Offload all video hosting to Azure Blob Storage; deliver assets using secure Shared Access Signature (SAS) tokens and leverage client-side caching.
Transaction Failures During Multi-Course Checkout	High	Wrap purchase and enrollment logic within the Unit of Work pattern to ensure strict database ACID transactions, preventing inconsistent states.
Tampering or Forgery of Completion Certificates	Medium	Issue each generated PDF with a cryptographically unique verification code and maintain a public verification endpoint to validate authenticity.

Identified Risk	Impact Level	Mitigation Strategy
Schedule Slippage Across Development Sprints	Medium	Conduct weekly sprint reviews aligned with project milestones; manage features via a priority matrix to safeguard critical path deliverables.

1.5 Key Performance Indicators (KPIs)

- **System Availability (Uptime):** Minimum 99.5% uptime on Microsoft Azure hosting.
- **Server Response Time:** Average page and API response latency under 1.5 seconds.
- **Video Playback Latency:** Stream buffering and playback startup under 2 seconds.
- **Payment Transaction Success Rate:** Greater than 99% seamless transaction completion rate without state mismatch.
- **Code Quality & Reliability:** Zero critical security issues across common vulnerabilities, and above 75% test coverage for critical business logic modules.

2. Literature Review

2.1 Domain & Technical Context

Modern educational ecosystems demand highly maintainable, loosely coupled architectures capable of supporting rich digital media and complex user workflows. Traditional monolithic learning platforms frequently suffer from tight coupling between business logic and database access, resulting in high maintenance overhead and scaling bottlenecks.

By utilizing ASP.NET Core MVC structured around an N-Tier architecture, EduHub decouples presentation, domain services, and data persistence. This approach isolates business rules from infrastructure concerns and aligns with enterprise software engineering standards.

2.2 Architectural Comparison & Design Patterns

Feature / Architecture	Traditional / Legacy Approach	EduHub Implementation
Structural Architecture	Monolithic single-tier architecture with mixed UI and database logic.	Strict N-Tier Architecture (Presentation, Business Logic, and Data Access layers).
Data Persistence	Direct database queries embedded in controller actions.	Generic Repository Pattern with Unit of Work over Entity Framework Core.
Pricing Strategy	Hard-coded branching statements for pricing and promotional rules.	**Strategy Pattern** for clean, pluggable pricing calculation and dynamic discounts.
Event & State Handling	Manual status updates across unrelated services.	**Observer Pattern** to listen for course completion events and trigger automated certificate issuance.
Media Delivery	Local server file storage, risking server disk exhaustion.	Scalable cloud object storage via Azure Blob Storage with secure SAS access.

2.3 Evaluation & Academic Standards Alignment

The design and execution of EduHub are structured to align directly with the Digital Egypt Pioneers Initiative (DEPI) grading criteria:

- **Engineering Standards:** Full compliance with SOLID design principles and clean coding practices.
- **Architecture Rigor:** Clear separation of concerns verified across class contracts and dependency injection configuration.
- **Security Standards:** Protection against common vulnerabilities (XSS, CSRF, and SQL Injection) backed by ASP.NET Identity authorization filters.

- **Comprehensive Documentation:** Complete traceability from stakeholder requirements to physical cloud deployment artifacts.

3. Requirements Gathering

3.1 Stakeholder Analysis

- **Student (Learner):** Primary end-user focused on finding quality courses, seamless checkout, distraction-free video playback, structured self-assessment via quizzes, and verified certification upon completion.
- **Instructor (Content Creator):** Domain expert requiring intuitive tools to structure curriculum, manage video content, configure flexible pricing, monitor enrollments, and track course earnings.
- **Administrator:** Platform manager responsible for auditing course quality before publication, managing user permissions, overseeing platform policies, and reviewing financial reports.
- **DEPI Evaluation Panel:** Technical reviewers assessing architectural discipline, code hygiene, security posture, and full-stack software delivery capabilities.

3.2 User Stories & Acceptance Criteria

User Story 1: Course Curriculum Exploration

- **As a** prospective Student,
- **I want to** search, filter, and preview available courses by category, level, and price,
- **So that** I can identify the most relevant material for my career development.
- **Acceptance Criteria:**
    - Users can search courses using keywords and filter by category or price range.
    - Course detail pages present instructor credentials, syllabi, ratings, and promotional videos.

User Story 2: Structured Course Authoring

- **As an** Instructor,
- **I want to** build courses organized by sections and lessons and upload high-definition video materials,

- **So that** I can deliver a structured learning experience to enrolled students.
- **Acceptance Criteria:**
    - Instructors can create, reorder, and edit sections and lessons within a unified builder interface.
    - Video uploads are processed directly to cloud storage with confirmed upload statuses.

User Story 3: Verified Certificate Issuance

- **As an** enrolled Student,
- **I want to** automatically receive a tamper-proof PDF certificate upon completing all lessons and passing required quizzes,
- **So that** I can showcase my verified credentials on professional platforms such as LinkedIn.
- **Acceptance Criteria:**
    - System marks course completion only when all module lessons are completed and quiz passing grades are achieved.
    - PDF certificate is generated automatically with student name, completion timestamp, and a verifiable unique identifier.

3.3 Functional Requirements

1. **Authentication & Role-Based Access Control (RBAC):**
    - Account registration, login, and profile management utilizing ASP.NET Identity.
    - Granular authorization roles: Student, Instructor, and Admin.
2. **Curriculum & Content Management:**
    - Full CRUD operations for courses, sections, and individual lessons.
    - Category and subcategory taxonomic hierarchy.
    - Draft, Pending Approval, and Published workflow states for quality control.
3. **Cloud Media Management:**
    - Cloud video file persistence via Azure Blob Storage.

- Responsive media player integration (Video.js) with secure, tokenized URL access.

4. **Learning & Progress Tracking:**
    - Dynamic lesson completion logging with percentage-based course progress bars.
    - Note-taking and lesson bookmarking during video playback.
5. **Assessment & Evaluation Engine:**
    - Multi-format quiz builder (multiple-choice, true/false) attached to curriculum milestones.
    - Automated grading engine with defined passing thresholds.
    - 5-star course rating and textual review system with moderation options.
6. **E-Commerce & Payment Engine:**
    - Multi-item shopping cart supporting single and bulk course purchases.
    - Integration with Stripe API for checkout and automated invoice generation.
    - Discount calculation service using the Strategy Pattern for promotional vouchers.
7. **Automated Document Generation:**
    - Server-side dynamic PDF compilation of certificates via iTextSharp.
    - Public validation module to authenticate certificate integrity using registration codes.
8. **Administration & Analytics Suite:**
    - Centralized administrative control panel for user moderation and course vetting.
    - Visual performance metrics and revenue graphs utilizing Chart.js.

3.4 Non-Functional Requirements

- **Performance & Responsiveness:**
    - Platform pages must load within 2 seconds under regular network conditions.

    - Database queries must incorporate eager loading and indexed lookups to prevent N+1 query overhead.

- **Security & Data Protection:**
    - Application must adhere to standard security best practices against OWASP Top 10 risks (including SQL Injection prevention via EF Core parameterization, Anti-Forgery Tokens for POST requests, and input sanitization).
    - Sensitive user credentials and checkout tokens must be encrypted in transit (HTTPS/TLS) and at rest.
- **Scalability & Modularity:**
    - Layered codebase built on N-Tier principles to permit independent scaling or future migration to microservices.
    - Stateless web application design to support future horizontal scaling across cloud instances.
- **Reliability & Consistency:**
    - Critical transactions (e.g., checkout and student enrollment) must execute through the Unit of Work pattern to ensure transactional integrity.
- **Usability & Accessibility:**
    - Fully responsive user interface across desktop, tablet, and mobile browsers built on Bootstrap 5.
    - Clean UI layout with standard contrast ratios and intuitive navigation flows.